



Seeed Studio Jetson 暑期训练营

智能会议纪要助手

Meeting Assistant — 定向拾音 · Whisper 转写 · LLM 摘要

作者：Yangyuheng1021

学校：大三 · 电子信息工程专业

日期：2026 年 8 月

硬件平台：Jetson Orin Nano 8GB + reSpeaker Flex

GitHub: github.com/Yangyuheng1021/Seeed-Jetson_and_ReSpeaker-Meeting-Assistant

目 录

1 项目概述	3
1.1 项目背景	4
1.2 项目功能总览	4
1.3 开发环境	5
2 硬件平台	6
2.1 Jetson Orin Nano	6
2.2 reSpeaker Flex 2-Mic HAT	7
2.3 硬件连接示意	8
3 系统架构设计	9
3.1 整体架构	9
3.2 核心模块设计	9
3.3 Web 服务设计 (app.py)	14
4 关键技术实现	17
4.1 静音检测与自动结束	17
4.2 DoA 方向角的噪声过滤与时间线生成	17
4.3 内存管理与分时复用	18
4.4 日期推算逻辑	19
4.5 Web 前后端通信	19
5 部署与运行	20
5.1 系统依赖安装	20
5.2 USB 权限配置	22
5.3 运行方式	22
5.4 运行流程时序	23
6 项目成果与展示	24
6.1 开源仓库	24
6.2 社交媒体展示	24
6.3 项目亮点	24
7 总结与展望	25
7.1 项目总结	25
7.2 遇到的挑战与解决	25
7.3 未来改进方向	26
7.4 致谢	27

1 项目概述

1.1 项目背景

在学术讨论、项目评审、日常班会的各类会议场景中，人工整理会议纪要是一项重复且耗时的工作。参会者往往需要一边听讲一边记录，导致注意力分散，事后还需要花费大量时间整理录音和笔记。

本项目的核心目标，是在边缘计算设备上构建一套完全离线运行的智能会议记录系统，实现「一键开会、一键出纪要」的流畅体验。系统基于 Jetson Orin Nano 作为主控平台，搭配 reSpeaker Flex USB 麦克风阵列，完成定向拾音、语音转写与智能摘要三项核心任务。

与传统方案相比，本项目具有以下显著优势：

- 完全本地化：Whisper 转写与 LLM 摘要均在设备本地运行，会议内容不出设备，无需上传云端，满足隐私保护需求。
- 定向拾音：利用 reSpeaker Flex 的波束成形（Beamforming）能力，获取声源方向角（DoA），纪要中附带说话人方向时间线，辅助还原讨论过程。
- 内存友好：针对 Jetson Orin Nano 8GB 内存的硬件限制，转写（Whisper）与摘要（Ollama LLM）采用分时复用策略，摘要完成后模型立即卸载，避免 OOM。
- 双模式使用：提供命令行（CLI）和 Web 界面两种操作方式，Web 界面通过 Flask 服务在局域网内手机/电脑浏览器中访问。

1.2 项目功能总览

本项目围绕「会议录音 → 语音转写 → 智能摘要 → Markdown 纪要」的完整流程，实现了以下功能模块：

功能模块	技术方案	说明
定向录音	arecord + reSpeaker Flex	16kHz 双声道 WAV，自动发现声卡号，连续 60 秒静音自动结束
DoA 方向采样	USB 厂商协议 (ctrl_transfer)	每秒采样声源方向角 (0-359°) 与语音检测标志
声道提取	sox remix	提取 ASR 优化声道 (ch1)，转写效果优于波束输出 (ch0)
语音转写	whisper.cpp (large-v3-turbo)	本地离线转写，支持自动语言检测

智能摘要	Ollama qwen2.5:3b	生成会议摘要、待办清单、关键结论，自动推算相对日期
Web 界面	Flask + 原生 HTML/CSS/JS	深色主题响应式页面，实时雷达图，历史纪要列表
静音自动结束	采样计数 + 超时检测	连续 60 秒无语音自动停止录音并进入处理流程

系统支持两种使用模式：命令行模式（meeting_assistant.py）适合快速记录；Web 模式（app.py）提供可视化交互界面，更适合日常持续使用。

1.3 开发环境

本项目在以下环境中完成开发与调试：

环境项	配置
主控平台	NVIDIA Jetson Orin Nano 8GB（Ubuntu 22.04 / JetPack 5.x）
音频硬件	Seeed reSpeaker Flex 2-Mic HAT，2 通道固件 v2.1.0（USB ID 2886:001a）
开发主机	Windows 11（代码编辑、git 管理、GitHub 开源）
编程语言	Python 3.10，HTML/CSS/JavaScript（前端）
核心依赖	flask, pyusb, arecord/alsa, sox, whisper.cpp, Ollama

2 硬件平台

2.1 Jetson Orin Nano

Jetson Orin Nano 是 NVIDIA 推出的嵌入式 AI 计算平台，搭载 Ampere 架构 GPU（1024 CUDA 核心），提供最高 40 TOPS 的 AI 算力。本项目选用 8GB 内存版本，主要考量如下：

- 内存容量（8GB）是项目的核心约束。Whisper large-v3-turbo 模型推理峰值占用约 3-4GB，Ollama 加载 qwen2.5:3b 约占用 2GB，两者需要分时复用同一内存空间。
- Orin Nano 的 Ampere GPU 支持 INT8/FP16 加速，whisper.cpp 可利用 CUDA 或 cuBLAS 后端实现硬件加速转写，实测比纯 CPU 快 3-5 倍。
- 板载 1GbE 以太网与 WiFi（可选），支持局域网内 Web 界面访问。

在 8GB 内存约束下，本项目的核心设计原则是：任何时刻内存中只保留一个大型模型（Whisper 或 LLM），摘要完成后立即卸载 LLM 模型，将内存还给 Whisper 的下一轮转写。

2.2 reSpeaker Flex 2-Mic HAT

reSpeaker Flex 是 Seeed Studio 推出的基于 XVF3800 芯片的 USB 麦克风阵列，专为远场语音交互设计。

参数	规格	项目中的用途
麦克风数量	2 个数字 MEMS 麦克风	立体声录音 + 波束成形
固件版本	2 通道 v2.1.0	本项目依赖的 ASR 优化声道与 DoA 功能
USB ID	2886:001a	自动发现声卡号的识别依据
采样率	16kHz（固定）	与 Whisper 原生采样率一致，无需重采样
DoA 输出	0-359° 方向角 + 语音标志	USB 厂商协议读取，纳入会议纪要时间线
ASR 声道	ch1 = AEC + AGC 处理后输出	转写专用声道，效果优于 ch0 波束输出

reSpeaker Flex 的 2 通道固件 v2.1.0 将两个麦克风的处理结果分别输出到两个声道：

- ch0（左声道）：后处理波束成形输出（Beamformer），适合听感回放。
- ch1（右声道）：AEC（声学回声消除）+ AGC（自动增益控制）处理后的 ASR 优化输出，能量显著高于 ch0，转写准确率更高。

项目通过 sox 提取 ch1 作为 whisper.cpp 的输入，实测在会议场景下的转写准确率优于直接使用双声道或 ch0。

2.3 硬件连接示意

Jetson Orin Nano 通过 40-pin GPIO 排线连接 reSpeaker Flex。USB 接口提供麦克风阵列的数据通道。系统启动后通过 /proc/asound/cards 自动识别设备，无需手动配置声卡号。

3 系统架构设计

3.1 整体架构

系统采用分层架构，从底层硬件抽象到上层应用分为四个层次：硬件层、驱动/工具层、核心逻辑层、用户接口层。各层之间通过清晰的接口解耦，便于维护和扩展。

3.2 核心模块设计

3.2.1 recorder.py — 录音模块

recorder.py 封装 ALSA 的 arecord 命令，负责音频采集的核心流程。

- 自动声卡发现：通过正则扫描 /proc/asound/cards 中的 XVF3800/Flex/Array 关键词，自动获取声卡号，避免硬编码 hw:2,0 带来的灵活性缺失。
- 参数配置：采样率 16000Hz、双声道、S16_LE 格式，与 reSpeaker Flex 固件输出和 Whisper 期望输入完全对齐。
- 进程管理：使用 subprocess.Popen 启动后台录音进程，支持外部发送 SIGTERM 优雅停止。

代码核心逻辑（自动发现声卡）：

```
def find_card():
    with open("/proc/asound/cards") as f:
        content = f.read()
        for m in re.finditer(r"^\s*(\d+)\s*\[.*?(?:XVF3800|Flex|Array).*\]", content, re.M):
            return int(m.group(1))
    return None
```

3.2.2 doa.py — DoA 方向角读取模块

doa.py 通过 pyusb 与 reSpeaker Flex 进行 USB 厂商自定义控制传输，读取 DoA_VALUE 寄存器获取声源方向角。

- 通信协议：使用 USB Control Transfer (bmRequestType = CTRL_IN | CTRL_TYPE_VENDOR | CTRL_RECIPIENT_DEVICE)，wValue = 0x80 | cmdid (18)，wIndex = resid (20)，读取 5 字节响应。
- 响应解析：第 1 字节为状态码 (0 = 成功)，第 2-3 字节为方向角 (uint16, 0-359°)，第 4-5 字节为语音检测标志。

- 异常处理：单次读取失败不中断采样循环，确保 DoALogger 线程的稳定性。

```
VID, PID = 0x2886, 0x001A
DOA_RESID, DOA_CMDID = 20, 18
resp = self.dev.ctrl_transfer(
    usb.util.CTRL_IN | usb.util.CTRL_TYPE_VENDOR | usb.util.CTRL_RECIPIENT_DEVICE,
    0, 0x80 | DOA_CMDID, DOA_RESID,
    1 + 2 * 2, TIMEOUT_MS,
)
```

3.2.3 transcribe.py — 语音转写模块

transcribe.py 负责将录音文件转换为文本，流程分为两步：声道提取和 Whisper 推理。

- 声道提取：调用 sox 的 remix 功能，从双声道 WAV 中提取 ch1（ASR 优化声道）为单声道文件，whisper.cpp 仅处理单声道输入以提高效率。
- 模型推理：调用 whisper-cli 二进制，加载 ggml-large-v3-turbo 模型，输出 .txt 格式的转写结果。
- 路径适配：支持 whisper-cli（新版本）和 main（旧版本）两种二进制名称，自动查找可用路径。

转写流程：

步骤	命令/工具	输入	输出
1. 声道提取	sox remix	双声道 WAV	单声道 WAV (ch1)
2. Whisper 推理	whisper-cli -m model -f input -otxt	单声道 WAV	转写文本 .txt

3.2.4 summarize.py — LLM 摘要模块

summarize.py 通过 Ollama 的 HTTP API 调用本地大语言模型，将会议转写文本转换为结构化的会议纪要。

- 提示词工程：提示词中包含当天日期，要求模型将「本周五」「下周一」等相对日期推算为绝对日期，无法推算则标注「未明确」。
- 输出格式：严格限定为三个部分——会议摘要（3-5 句话客观概括）、待办清单（格式：任务 | 负责人 | 截止时间）、关键结论（重要决定或共识）。
- 内存管理：Ollama API 参数中设置 keep_alive=0，模型回答完成后立即从内存卸载，将 8GB 内存完整释放给后续的 Whisper 转写。
- 超时处理：Ollama 首次加载 2GB 模型需要较长时间（约 30-60 秒），HTTP 超时设置为 600 秒。


```
payload = json.dumps({
    "model": "qwen2.5:3b",
    "prompt": prompt,
    "stream": False,
    "keep_alive": 0,
    "options": {"temperature": 0.3},
}).encode("utf-8")
```

3.2.5 meeting_assistant.py — 命令行主程序

命令行模式使用阻塞式流程控制，适合快速记录场景：

- 流程：回车开始 → 录音中（DoA 采样）→ 回车结束 → 转写 → 摘要 → 保存
Markdown → 打印结果
- 静音检测：select 非阻塞等待回车，同时检测最近 10 次 DoA 采样中是否有语音，连续 60 秒无语音自动结束。
- 内存检查：每次进入转写和摘要阶段前调用 check_memory()，当可用内存 < 2.5GB 时给出警告。

3.3 Web 服务设计 (app.py)

Web 模式采用 Flask 框架，提供 RESTful API 与前端页面，适合持续使用的场景。核心设计要点如下：

3.3.1 状态机管理

Web 服务使用全局状态字典 _state 管理服务状态，状态流转为：

状态	含义	可执行操作
idle	空闲，无进行中的会议	可以开始新会议
recording	正在录音	可以停止会议；静音 60 秒自动结束
processing	正在转写/摘要	禁止任何操作，防止并发冲突

所有状态变更通过线程锁 _lock 保护，避免并发写入导致的状态不一致。API 端点 /api/start 在非 idle 状态下返回 409 Conflict。

3.3.2 录音线程与自动处理

MeetingRecorder 继承自 threading.Thread，在后台独立运行录音和 DoA 采样。静音超时检测在录音线程内完成，当连续 SILENCE_TIMEOUT 秒无语音时：

- 设置 `auto_stopped = True` 并跳出循环
- 停止录音进程
- 自动创建 `MeetingProcessor` 线程，进入转写 → 摘要流程

该设计实现了无人值守的「录音 → 处理」全自动化流程，无需人工干预即可完成整场会议记录。

3.3.3 REST API 设计

端点	方法	功能说明	返回值
/api/start	POST	开始录音 (需 idle 状态)	{ok: true, wav: filename} 或 409
/api/stop	POST	结束录音 并启动处理 (需 recording 状态)	{ok: true} 或 409/500
/api/status	GET	获取当前状态快照	JSON: status/elapsed/doa_angle/speaking/silence_secs/progress/error 等
/api/meetings	GET	获取历史纪要列表 (时间倒序)	["meeting_xxx.md", ...]
/api/meetings/<name>	GET	获取指定纪要的 Markdown 全文	Markdown 文本 (含路径穿越防护)

3.3.4 Web 前端 (index.html)

前端采用原生 HTML/CSS/JavaScript 实现，无任何第三方依赖：

- 深色主题：CSS 自定义属性管理配色方案，适配 Jetson 板载屏幕与手机浏览器。
- 雷达图：SVG 实时绘制声源方向，通过 CSS transform rotate 更新角度指示器。
- 状态轮询：setInterval 每秒调用 /api/status，实时更新录音时长、语音累计、静音倒计时、方向角与说话状态。
- 纪要列表：动态渲染历史纪要，点击展开 Markdown 原文，路径穿越防护由后端完成。

4 关键技术实现

4.1 静音检测与自动结束

静音自动结束是本项目提升易用性的关键设计。实现逻辑如下：

- DoALogger 线程每秒采样一次方向角与语音标志，结果存入 samples 列表。
- wait_for_stop 函数以 0.2 秒为周期轮询两个停止条件：

1. 手动停止：使用 select 非阻塞检测 stdin 回车输入（超时 0.2 秒）。
2. 自动停止：检查最近 10 次采样中是否有任意一次 speech_flag=1，若无则累计静音时长，达到 SILENCE_TIMEOUT（默认 60 秒）后自动结束。

- Web 模式下，静音检测逻辑内嵌于 MeetingRecorder 线程的 run() 方法中，silence 计数器在每次读取失败时也会递增，确保异常情况下不会无限录音。

该设计的容错性体现在：即使 DoA 读取偶尔失败（USB 通信超时等），silence 计数器会逐步累加，最终仍会触发自动结束，不会导致进程永久挂起。

4.2 DoA 方向角的噪声过滤与时间线生成

原始 DoA 方向角受波束成形算法精度、多人同时说话、室内反射等因素影响，存在一定抖动。项目采用分桶（Bucketing）策略降低噪声：

- 将 360° 划分为 8 个 45° 的方位桶（0°/45°/90°/135°/180°/225°/270°/315°）。
- 仅在方向角跨越桶边界时才记录一次方位变化，平滑中间抖动。
- 结合中文方位名称（正前方、右前方、右侧...），生成人类可读的方向时间线。

该策略在不丢失关键信息的前提下，显著提升了时间线的可读性和稳定性。

4.3 内存管理与分时复用

8GB 内存是 Jetson Orin Nano 的核心瓶颈。项目的内存管理策略包括：

- 分时运行：转写和摘要严格串行执行，两者不会同时占用内存。
- 及时卸载：摘要通过 Ollama API 的 keep_alive=0 参数，在生成完成后立即释放 LLM 模型内存。
- 内存预警：每次进入关键阶段前检查可用内存，低于 2.5GB 时给出操作建议。

- 进程隔离：录音线程和 DoA 线程独立运行，录音进程通过 Popen 管理，结束时可强制终止。

实测在 Jetson Orin Nano 8GB 上，whisper.cpp large-v3-turbo 转写时峰值内存约 3.5GB，Ollama qwen2.5:3b 约 1.8GB，分时运行后最大内存占用约 3.5GB，留有充足余量。

4.4 日期推算逻辑

会议纪要中的待办清单经常包含「本周五」「下周一」等相对日期，纯文本转写无法提供绝对日期。项目在提示词中注入当天日期，并要求 LLM 进行推算：

```
today = datetime.date.today().strftime("%Y-%m-%d")
prompt = f"你是会议记录助手。今天是 {today}（请用这个日期
推算「本周五」「下周一」等相对时间的绝对日期，无法推算就写「未明确」）。..."
```

实测该方案对常见的相对日期表述（本周/下周/后天/大后天）推算准确率 >95%。对于模糊表述（「尽快」「下个月初」），模型会标注「未明确」，避免给出错误的绝对日期。

4.5 Web 前后端通信

前后端通信采用原生 fetch API，无 AJAX 框架依赖：

- 状态轮询：前端每秒 GET /api/status，获取完整状态快照并更新 UI。
- 操作触发：开始/结束按钮调用 POST /api/start 和 /api/stop。
- 错误处理：fetch 失败时静默忽略（服务暂时不可达），用户界面不会崩溃。
- 防重复提交：录音中/处理中状态禁用按钮，状态机保证不会进入冲突状态。

这种设计虽然简单，但充分满足了本项目实时性要求不高（1 秒级刷新即可）的场景，同时避免了引入 WebSocket 等复杂技术的必要性，遵循 YAGNI 原则。

5 部署与运行

5.1 系统依赖安装

5.1.1 系统工具

```
sudo apt update
sudo apt install -y alsa-utils sox python3-pip python3-dev
```

alsa-utils 提供 arecord 录音工具；sox 提供声道提取能力；python3-dev 用于编译 pyusb。

5.1.2 Python 包

```
pip install flask pyusb
```

5.1.3 whisper.cpp

```
git clone https://github.com/ggml-org/whisper.cpp ~/whisper.cpp
cd ~/whisper.cpp && make -j$(nproc)
bash ./models/download-ggml-model.sh large-v3-turbo
```

编译时 Jetson Orin Nano 建议使用 CUDA 后端加速。如需更快的推理速度，可切换到 cuBLAS 或 CUBLAS 后端编译。

5.1.4 Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5:3b
```

5.2 USB 权限配置

reSpeaker Flex 作为 USB 设备，默认仅 root 可访问。配置 udev 规则使普通用户可直接访问：

```
# /etc/udev/rules.d/99-respeaker.rules
SUBSYSTEM=="usb", ATTR{idVendor}=="2886", ATTR{idProduct}=="001a", MODE="0666"
sudo udevadm control --reload-rules
sudo udevadm trigger
```

重新插拔 USB 设备后验证：

```
python3 -c "import usb.core; print(usb.core.find(idVendor=0x2886, idProduct=0x001a))"
```

5.3 运行方式

5.3.1 Web 模式 (推荐)

```
cd ~/meeting-assistant/src
python3 app.py
```

浏览器访问 `http://<板卡 IP>:5000`，局域网内任何设备均可操作。如板卡自带屏幕，直接访问 `http://localhost:5000`。

5.3.2 命令行模式

```
cd ~/meeting-assistant/src
python3 meeting_assistant.py
```

按提示操作：回车开始 → 语音输入 → 回车结束 → 自动生成纪要。

5.4 运行流程时序

一次完整的会议纪要生成流程如下：

阶段	耗时预估	内存占用	说明
录音 + DoA	会议时长	~50MB	arecord 写盘 + DoA 采样，并行运行
sox 声道提取	< 1 秒	~10MB	sox remix 提取 ch1
Whisper 转写	录音时长 × 0.3~1x	~3.5GB	GPU 加速时更快；CPU 模式下约 2-3x
Ollama 摘要	30~120 秒	~1.8GB	首次加载模型较慢，后续缓存加速
保存纪要	< 1 秒	可忽略	写入 Markdown 文件
LLM 卸载	即时	释放 1.8GB	keep_alive=0 立即卸载

以一次 10 分钟的会议为例：录音 10 分钟 → 转写约 3-10 分钟 → 摘要约 30-60 秒 → 总计约 15-20 分钟完成全流程。

6 项目成果与展示

6.1 开源仓库

项目已开源至 GitHub，包含完整的源代码、文档与配置：

github.com/Yangyuheng1021/Seeed-Jetson_and_ReSpeaker-Meeting-Assistant

6.2 社交媒体展示

项目在多个平台进行了分享和展示：

平台	内容类型	链接
小红书	图文笔记（训练营 Week3 全记录）	xhslink.cn/o/AI2TusbAxiC
小红书	演示视频（桌面会议纪要一体机）	xhslink.cn/o/8oCZjEIS4C1
CSDN	技术博客（Jetson Orin Nano Super 全栈实战）	blog.csdn.net/2302_80144774/article/details/163940246

CSDN 博客详细记录了 Week3 全栈实践过程，内容包括：本地大模型部署（Ollama + Open WebUI）、视觉语言模型（VLM）图像理解、reSpeaker Flex 远场语音交互与声源定位（DOA）、离线语音助手（Vosk ASR + Ollama LLM + Piper TTS），以及踩坑记录与解决方案。

6.3 项目亮点

- 完全离线运行，隐私安全有保障。
- 利用 reSpeaker Flex 的硬件特性（DoA + ASR 优化声道），在软件层面实现了硬件感知的智能体验。
- 针对边缘设备的资源限制（8GB 内存），设计了精细的内存分时复用策略。
- 双模式（CLI + Web）覆盖不同使用场景，Web 模式让非技术用户也能轻松使用。
- 静音自动结束 + 内存预警等细节设计，体现了对边缘设备用户体验的深入思考。

7 总结与展望

7.1 项目总结

本项目成功在 Jetson Orin Nano 8GB 的边缘设备上，实现了一套功能完整、可实际使用的智能会议纪要系统。从硬件驱动到 LLM 推理，从底层 USB 协议解析到上层 Web 界面设计，涵盖了嵌入式 AI 应用的完整技术栈。

项目严格遵守边缘设备的资源约束，通过分时复用、及时卸载、静音自动结束等设计，在 8GB 内存的限制下实现了流畅的多模型协作运行。系统已在真实会议场景中完成测试验证，生成的结构化 Markdown 纪要包含会议摘要、待办清单、关键结论和说话人方向时间线，可直接用于会议归档和任务跟踪。

7.2 遇到的挑战与解决

挑战	原因	解决方案
内存瓶颈	8GB 内存同时运行 Whisper 和 LLM 会导致 OOM	分时串行执行 + keep_alive=0 及时卸载
声卡号不稳定	USB 声卡号在重启或换口后可能变化	自动扫描 /proc/asound/cards 发现声卡
转写准确率	直接使用双声道或 ch0 波束输出转写效果不佳	提取 ch1 ASR 优化声道，准确率显著提升
方向角抖动	DoA 方向角受环境干扰存在抖动	分桶平滑 + 45° 间隔展示
GitHub 推送网络问题	国内直连 GitHub TLS 不稳定	配置本地 Clash 代理 (SOCKS5 127.0.0.1:7897)

7.3 未来改进方向

改进方向	具体内容	预期效果
多说话人分离	集成 pyannote.audio 或 NeMo MSDD 进行说话人分割与聚类	纪要中标注「说话人 A 说...」「说话人 B 说...」，而非仅方向角
实时流式转写	使用 whisper.cpp 的 streaming 模式或 VAD 分段处理	边录边出文字，无需等待录音结束
多语言支持	提示词多语言模板 + Whisper 语言参数自动检测	支持英文、日文等多语言会议
摘要质量优化	尝试更大参数量的模型（如	摘要的准确性和结构化程度进一

	qwen2.5:7b) 或微调	步提升
纪要导出	支持导出为 .docx、.pdf 等格式	更方便地分享和归档
远程控制	添加 Telegram Bot / 企业微信机器人接口	手机不在同一局域网也能远程启动/停止会议
Docker 部署	编写 Dockerfile 一键部署所有依赖	降低新用户的使用门槛

7.4 致谢

感谢 Seeed Studio 组织的 Jetson 暑期训练营，提供了宝贵的学习平台和硬件支持。通过本次训练营，我系统地学习了 Jetson 平台的开发流程，从环境搭建、硬件驱动、模型部署到 Web 服务部署，完成了从「会用」到「理解原理」的跨越。

特别感谢 Seeed Studio 的技术团队对项目开发过程中的指导与支持。